

Математическое моделирование

Лабораторная работа № 5

Сергей Павленко

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Теоретические сведения	8
3.2	Задача	9
4	Модель хищник-жертва: численное решение и визуализация	11
4.1	Инициализация проекта и загрузка пакетов	11
4.2	Начальные условия и временной интервал	12
4.3	Параметры модели	12
4.4	Определение модели	12
4.5	Утилита: один прогон (решение, таблица, графики, сохранение) .	13
4.6	Запуск модели	15
4.7	Вывод первых строк таблицы	15
4.8	Показать график в интерактивной среде	15
5	Параметрическое исследование модели хищник-жертва	16
5.1	Активация проекта и загрузка пакетов	16
5.2	Установка каталогов	16
5.3	Определение моделей	17
5.4	Базовые параметры в Dict	17
5.5	Функция-обертка для запуска одного эксперимента	18
5.6	Запуск базового эксперимента	21
5.7	Визуализация базового эксперимента	22
5.8	Фазовый портрет базового эксперимента	22
5.9	Второй базовый эксперимент (расширенная система)	23
5.10	Параметрическое сканирование	25
5.11	Запуск всех экспериментов и сбор результатов	26
5.12	Анализ результатов сканирования	28
5.13	Сравнительный график $x(t)$	28
5.14	Сравнительный график $y(t)$	30
5.15	Сравнительный фазовый портрет	31
5.16	График зависимости norm_final от параметров	32
5.17	Бенчмаркинг	33

5.18 График времени вычисления	35
5.19 Сохранение всех результатов	36
5.20 Базовые эксперименты	37
5.21 Параметрическое сканирование	41
5.22 Анализ метрики norm_final	44
5.23 Время вычислений	45
5.24 Выводы	45
Список литературы	47

Список иллюстраций

5.1	Базовая модель: зависимость $x(t)$, $y(t)$	37
5.2	Базовая модель: фазовый портрет	38
5.3	Расширенная модель: зависимость $x(t)$, $y(t)$	39
5.4	Расширенная модель: фазовый портрет	40
5.5	Сканирование траекторий $x(t)$	41
5.6	Сканирование траекторий $y(t)$	42
5.7	Сканирование фазовых траекторий	43
5.8	Зависимость norm_final	44
5.9	Зависимость времени вычисления	45

Список таблиц

1 Цель работы

Освоить и проанализировать математическую модель взаимодействия популяций типа «хищник–жертва».

2 Задание

1. Построить фазовую зависимость $x(y)$, а также временные зависимости $x(t)$ и $y(t)$
2. Определить стационарные состояния системы

3 Выполнение лабораторной работы

3.1 Теоретические сведения

В работе исследуется классическая модель взаимодействия двух популяций — хищников и жертв.

Обозначим численности популяций как X (хищники) и Y (жертвы). Предполагается, что система удовлетворяет следующим условиям (модель Лотки–Вольтерры):

1. Численности зависят только от времени, без учёта пространственного распределения
2. При отсутствии взаимодействия динамика описывается моделью Мальтуса: жертвы растут, хищники вымирают
3. Естественная смертность жертв и естественное размножение хищников не учитываются
4. Ограничение роста популяций отсутствует
5. Взаимодействие приводит к уменьшению численности жертв пропорционально числу хищников

Математическая запись модели:

$$\begin{cases} \frac{dx}{dt} = -ax(t) + bx(t)y(t) \\ \frac{dy}{dt} = cy(t) - dx(t)y(t) \end{cases}$$

где: - a — коэффициент убывания хищников

- b — коэффициент увеличения хищников

- c — коэффициент роста жертв

- d — коэффициент гибели жертв

Система может находиться в равновесии, если отсутствуют изменения во времени:

$$\frac{dx}{dt} = 0, \quad \frac{dy}{dt} = 0$$

При условии $x > 0, y > 0$ стационарная точка определяется как:

$$x_0 = \frac{a}{b}, \quad y_0 = \frac{c}{d}$$

3.2 Задача

Рассматривается система:

$$\begin{cases} \frac{dx}{dt} = -0.25x(t) + 0.025x(t)y(t) \\ \frac{dy}{dt} = 0.45y(t) - 0.045x(t)y(t) \end{cases}$$

Требуется: - построить графики $x(t)$ и $y(t)$

- построить фазовую траекторию

- определить равновесное состояние

Начальные условия: $x_0 = 8, y_0 = 11$

Стационарная точка:

$$x_0 = \frac{a}{b} = 10, \quad y_0 = \frac{c}{d} = 10$$

Для численного решения применялись внешние программные модули:

4 Модель хищник-жертва: численное решение и визуализация

Цель: решить систему ОДУ Лотки–Вольтерры, построить графики изменения популяций во времени, построить фазовый портрет, сохранить изображения и таблицу с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using CSV
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Начальные условия и временной интервал

```
x0 = 8.0
y0 = 11.0

u0 = [x0, y0]

t0 = 0.0
tmax = 150.0
tspan = (t0, tmax)
```

4.3 Параметры модели

```
a = 0.25
b = 0.025
c = 0.45
d = 0.045

p = (a = a, b = b, c = c, d = d)
```

4.4 Определение модели

$x(t)$ — численность жертв $y(t)$ — численность хищников

$$\frac{dx}{dt} = -ax + bxy \quad \frac{dy}{dt} = cy - dxy$$

```
function predator_prey!(dy, y, p, t)
    dy[1] = -p.a * y[1] + p.b * y[1] * y[2]
```

```
dy[2] = p.c * y[2] - p.d * y[1] * y[2]
end
```

4.5 Утилита: один прогон (решение, таблица, графики, сохранение)

```
function run_case(case_name; model!, u0, tspan, p, nt=1000)
```

— Сетка по времени —

```
tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
prob = ODEProblem(model!, u0, tspan, p)
sol = solve(prob, Tsit5(), saveat=tgrid)
```

— Таблица с результатами —

```
df = DataFrame(
    t = sol.t,
    x = first.(sol.u),
    y = last.(sol.u)
)
```

— График $x(t)$ и $y(t)$ —

```
plt_time = plot(
    sol,
    xlabel = "t",
```

```

    ylabel = "Численность",
    label = ["x(t) – жертвы" "y(t) – хищники"],
    lw = 2,
    title = "Динамика популяций – $case_name",
    legend = :topright
)

```

— Фазовый портрет —

```

plt_phase = plot(
    sol,
    idxs = (1, 2),
    xlabel = "x",
    ylabel = "y",
    lw = 2,
    label = "Фазовая траектория",
    title = "Фазовый портрет – $case_name",
    legend = :topright
)

```

— Сохранение графиков —

```

savefig(plt_time, plotsdir(script_name, "$(case_name)_time.png"))
savefig(plt_phase, plotsdir(script_name, "$(case_name)_phase.png"))

```

— Сохранение таблицы —

```

CSV.write(datadir(script_name, "table_$(case_name).csv"), df)

```

— Сохранение данных в JLD2 —

```

    @save datadir(script_name, "data_$(case_name).jld2") df p u0 tspan nt

    return (sol = sol, df = df, plt_time = plt_time, plt_phase = plt_phase)
end

```

4.6 Запуск модели

```

res = run_case(
    "predator_preymodel! = predator_preymodel!,
    u0 = u0,
    tspan = tspan,
    p = p,
    nt = 1000
)

```

4.7 Вывод первых строк таблицы

```

println("Первые 5 строк таблицы результатов:")
println(first(res.df, 5))

```

4.8 Показать график в интерактивной среде

```

res.plt_time

```

5 Параметрическое исследование модели хищник-жертва

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
using CSV
```

5.2 Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```


5.3 Определение моделей

Первая система (базовая модель Лотки–Вольтерры): $dx/dt = -ax + bxy$ $dy/dt = cy - dxy$

```
function syst_base!(dy, y, p, t)
    dy[1] = -p.a * y[1] + p.b * y[1] * y[2]
    dy[2] = p.c * y[2] - p.d * y[1] * y[2]
end
```

Вторая система (усложнённая модель с внутривидовым ограничением жертв): $dx/dt = -ax + bxy - kx^2$ $dy/dt = cy - dx*y$

```
function syst_extended!(dy, y, p, t)
    dy[1] = -p.a * y[1] + p.b * y[1] * y[2] - p.k * y[1]^2
    dy[2] = p.c * y[2] - p.d * y[1] * y[2]
end
```

5.4 Базовые параметры в Dict

model_type управляет выбором системы: :base -> классическая модель
:extended -> модель с дополнительным нелинейным членом $-k*x^2$

```
base_params = Dict(
    :x0 => 8.0,
    :y0 => 11.0,

    :tspan => (0.0, 150.0),
    :nt => 1000,

    :model_type => :base,      # :base или :extended
)
```

Параметры базовой модели

```
:a => 0.25,  
:b => 0.025,  
:c => 0.45,  
:d => 0.045,
```

Дополнительный параметр расширенной модели

```
:k => 0.002,  
  
:solver => Tsit5(),  
:experiment_name => "base_experiment"  
)  
  
println("Базовые параметры эксперимента:")  
for (key, value) in base_params  
    println(" $key = $value")  
end
```

5.5 Функция-обертка для запуска одного эксперимента

```
function run_single_experiment(params::Dict)  
    x0 = params[:x0]  
    y0 = params[:y0]  
    tspan = params[:tspan]  
    nt = params[:nt]
```

```

model_type = params[:model_type]
a = params[:a]
b = params[:b]
c = params[:c]
d = params[:d]
k = params[:k]
solver = params[:solver]

u0 = [x0, y0]
tgrid = collect(LinRange(tspan[1], tspan[2], nt))

if model_type == :base
    p = (a = a, b = b, c = c, d = d)
    prob = ODEProblem(syst_base!, u0, tspan, p)
else
    p = (a = a, b = b, c = c, d = d, k = k)
    prob = ODEProblem(syst_extended!, u0, tspan, p)
end

sol = solve(prob, solver; saveat = tgrid)

x_vals = first(sol.u)
y_vals = last(sol.u)

```

— Мини-анализ —

```

x_final = x_vals[end]
y_final = y_vals[end]

```

```

x_max = maximum(x_vals)
y_max = maximum(y_vals)

x_min = minimum(x_vals)
y_min = minimum(y_vals)

x_mean = mean(x_vals)
y_mean = mean(y_vals)

norm_final = sqrt(x_final^2 + y_final^2)

return Dict(
    "solution" => sol,
    "t_points" => sol.t,
    "x_values" => x_vals,
    "y_values" => y_vals,

    "x_final" => x_final,
    "y_final" => y_final,
    "x_max" => x_max,
    "y_max" => y_max,
    "x_min" => x_min,
    "y_min" => y_min,
    "x_mean" => x_mean,
    "y_mean" => y_mean,
    "norm_final" => norm_final,

    "parameters" => params

```

```
)  
end
```

5.6 Запуск базового эксперимента

```
data_base, path_base = produce_or_load(  
    datadir(script_name, "single"),  
    base_params,  
    run_single_experiment;  
    prefix = "ode",  
    tag = false,  
    verbose = true  
)  
  
println("\nРезультаты базового эксперимента:")  
println(" x_final: ", data_base["x_final"])  
println(" y_final: ", data_base["y_final"])  
println(" x_max: ", data_base["x_max"])  
println(" y_max: ", data_base["y_max"])  
println(" x_min: ", data_base["x_min"])  
println(" y_min: ", data_base["y_min"])  
println(" x_mean: ", round(data_base["x_mean"]; digits = 4))  
println(" y_mean: ", round(data_base["y_mean"]; digits = 4))  
println(" norm_final: ", round(data_base["norm_final"]; digits = 4))  
println(" Файл результатов: ", path_base)
```

5.7 Визуализация базового эксперимента

```
p1 = plot(
    data_base["t_points"], data_base["x_values"],
    label = "x(t) – жертвы",
    xlabel = "t",
    ylabel = "Численность",
    title = "Базовый эксперимент (model_type=$(base_params[:model_type]))",
    lw = 2,
    legend = :topright,
    grid = true
)
plot!(
    p1,
    data_base["t_points"], data_base["y_values"],
    label = "y(t) – хищники",
    lw = 2
)

savefig(p1, plotsdir(script_name, "single_experiment_base.png"))
```

5.8 Фазовый портрет базового эксперимента

```
p2 = plot(
    data_base["x_values"], data_base["y_values"],
    xlabel = "x",
    ylabel = "y",
```

```

    label = "Фазовая траектория",
    title = "Фазовый портрет (model_type=$(base_params[:model_type]))",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p2, plotsdir(script_name, "single_experiment_base_phase.png"))

```

5.9 Второй базовый эксперимент (расширенная система)

```

base_params2 = copy(base_params)
base_params2[:model_type] = :extended
base_params2[:experiment_name] = "base_experiment_extended"

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты второго базового эксперимента:")
println(" x_final: ", data_base2["x_final"])

```

```

println(" y_final: ", data_base2["y_final"])
println(" x_max: ", data_base2["x_max"])
println(" y_max: ", data_base2["y_max"])
println(" x_min: ", data_base2["x_min"])
println(" y_min: ", data_base2["y_min"])
println(" x_mean: ", round(data_base2["x_mean"]; digits = 4))
println(" y_mean: ", round(data_base2["y_mean"]; digits = 4))
println(" norm_final: ", round(data_base2["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base2)

p3 = plot(
  data_base2["t_points"], data_base2["x_values"],
  label = "x(t) – жертвы",
  xlabel = "t",
  ylabel = "Численность",
  title = "Базовый эксперимент (model_type=$(base_params2[:model_type]))",
  lw = 2,
  legend = :topright,
  grid = true
)
plot!(
  p3,
  data_base2["t_points"], data_base2["y_values"],
  label = "y(t) – хищники",
  lw = 2
)

savefig(p3, plotsdir(script_name, "single_experiment_extended.png"))

```



```

p4 = plot(
    data_base2["x_values"], data_base2["y_values"],
    xlabel = "x",
    ylabel = "y",
    label = "Фазовая траектория",
    title = "Фазовый портрет (model_type=$(base_params2[:model_type]))",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p4, plotsdir(script_name, "single_experiment_extended_phase.png"))

```

5.10 Параметрическое сканирование

Сканируем: - для базовой модели параметр a - для расширенной модели параметр k

```

param_grid = Dict(
    :x0 => [8.0],
    :y0 => [11.0],

    :tspan => [(0.0, 150.0)],
    :nt => [1000],

    :model_type => [:base, :extended],

    :a => [0.15, 0.25, 0.35, 0.50],

```

```

:b => [0.025],
:c => [0.45],
:d => [0.045],

:k => [0.0, 0.001, 0.002, 0.005],

:solver => [Tsit5()],
:experiment_name => ["parametric_scan"]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые a: ", param_grid[:a])
println("Исследуемые k: ", param_grid[:k])
println("="^60)

```

5.11 Запуск всех экспериментов и сбор результатов

```

all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Пореек: $i/$(length(all_params)) | model_type=$(params[:model_type]) | ")

```

```

data, path = produce_or_load(
    datadir(script_name, "parametric_scan"),
    params,
    run_single_experiment;
    prefix = "scan",
    tag = false,
    verbose = false
)

result_summary = merge(
    params,
    Dict(
        :x_final => data["x_final"],
        :y_final => data["y_final"],
        :x_max => data["x_max"],
        :y_max => data["y_max"],
        :x_min => data["x_min"],
        :y_min => data["y_min"],
        :x_mean => data["x_mean"],
        :y_mean => data["y_mean"],
        :norm_final => data["norm_final"],
        :filepath => path
    )
)

push!(all_results, result_summary)

df = DataFrame(
    t = data["t_points"],

```

```

        x = data["x_values"],
        y = data["y_values"],
        model_type = fill(string(params[:model_type]), length(data["t_points"])),
        a = fill(params[:a], length(data["t_points"])),
        k = fill(params[:k], length(data["t_points"]))
    )
    push!(all_dfs, df)
end

```

5.12 Анализ результатов сканирования

```

results_df = DataFrame(all_results)

println("\nСводная таблица результатов (первые 10 строк):")
println(first(results_df, 10))

CSV.write(datadir(script_name, "results_summary.csv"), results_df)

```

5.13 Сравнительный график $x(t)$

```

p5 = plot(size = (950, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
    )
end

```

```

    prefix = "scan",
    tag = false,
    verbose = false
)

label_text = params[:model_type] == :base ?
    "base, a=$(params[:a])" :
    "extended, k=$(params[:k])"

plot!(
    p5,
    data["t_points"], data["x_values"],
    label = label_text,
    lw = 2,
    alpha = 0.8
)
end
plot!(
    p5,
    xlabel = "t",
    ylabel = "x(t)",
    title = "Сканирование: траектории x(t) для разных параметров",
    legend = :outerright,
    grid = true
)
savefig(p5, plotsdir(script_name, "parametric_scan_x_comparison.png"))

```

5.14 Сравнительный график $y(t)$

```
p6 = plot(size = (950, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :base ?
        "base, a=$(params[:a])" :
        "extended, k=$(params[:k])"

    plot!(
        p6,
        data["t_points"], data["y_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end
plot!(
    p6,
    xlabel = "t",
```

```

ylabel = "y(t)",
title = "Сканирование: траектории y(t) для разных параметров",
legend = :outright,
grid = true
)
savefig(p6, plotsdir(script_name, "parametric_scan_y_comparison.png"))

```

5.15 Сравнительный фазовый портрет

```

p7 = plot(size = (950, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :base ?
        "base, a=$(params[:a])" :
        "extended, k=$(params[:k])"

    plot!(
        p7,
        data["x_values"], data["y_values"],

```

```

        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end
plot!(
    p7,
    xlabel = "x",
    ylabel = "y",
    title = "Сканирование: фазовые траектории",
    legend = :outerright,
    grid = true
)
savefig(p7, plotsdir(script_name, "parametric_scan_phase_comparison.png"))

```

5.16 График зависимости `norm_final` от параметров

```

p8 = plot(size = (900, 520), dpi = 150)

base_sub = results_df[results_df.model_type .== :base, :]
extended_sub = results_df[results_df.model_type .== :extended, :]

if nrow(base_sub) > 0
    plot!(
        p8,
        base_sub.a, base_sub.norm_final,
        seriestype = :scatter,
    )
end

```



```

        label = "base"
    )
end

if nrow(extended_sub) > 0
    plot!(
        p8,
        extended_sub.k, extended_sub.norm_final,
        seriestype = :scatter,
        label = "extended"
    )
end

plot!(
    p8,
    xlabel = "Параметр модели",
    ylabel = "norm_final",
    title = "Зависимость norm_final от параметров модели",
    legend = :topleft,
    grid = true
)
savefig(p8, plotsdir(script_name, "norm_final_vs_parameter.png"))

```

5.17 Бенчмаркинг

```

println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")

```

```

println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        u0 = [params[:x0], params[:y0]]

        if params[:model_type] == :base
            p = (a = params[:a], b = params[:b], c = params[:c], d = params[:d])
            prob = ODEProblem(syst_base!, u0, params[:tspan], p)
        else
            p = (a = params[:a], b = params[:b], c = params[:c], d = params[:d], k =
            prob = ODEProblem(syst_extended!, u0, params[:tspan], p)
        end

        return solve(
            prob,
            params[:solver];
            saveat = LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
        )
    end

    println("\nБенчмарк для model_type=$(params[:model_type]), a=$(params[:a]), k=$(p
    bmark = @benchmark $benchmark_run() samples = 80 evals = 1
    tsec = median(bmark).time / 1e9
    println(" Медианное время: ", round(tsec; digits = 6), " сек")

```

```

    push!(benchmark_results, (
        model_type = string(params[:model_type]),
        a = params[:a],
        k = params[:k],
        time = tsec
    ))
end

bench_df = DataFrame(benchmark_results)
CSV.write(datadir(script_name, "benchmark_results.csv"), bench_df)

```

5.18 График времени вычисления

```

p9 = plot(size = (900, 520), dpi = 150)

bench_base = bench_df[bench_df.model_type .== "base", :]
bench_extended = bench_df[bench_df.model_type .== "extended", :]

if nrow(bench_base) > 0
    plot!(
        p9,
        bench_base.a, bench_base.time,
        seriestype = :scatter,
        label = "base"
    )
end

```

```

if nrow(bench_extended) > 0
    plot!(
        p9,
        bench_extended.k, bench_extended.time,
        seriestype = :scatter,
        label = "extended"
    )
end

plot!(
    p9,
    xlabel = "Параметр модели",
    ylabel = "Время вычисления, сек",
    title = "Зависимость времени решения ODE от параметров модели",
    legend = :topleft,
    grid = true
)

savefig(p9, plotsdir(script_name, "computation_time_vs_parameter.png"))

```

5.19 Сохранение всех результатов

```

@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4 p5 p6 p7 p8 p9

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)

```

```
println("\nРезультаты сохранены в:")
println(" • data/${script_name}/single/ - базовые эксперименты")
println(" • data/${script_name}/parametric_scan/ - параметрическое сканирование")
println(" • data/${script_name}/results_summary.csv - сводная таблица")
println(" • data/${script_name}/benchmark_results.csv - результаты бенчмаркинга")
println(" • data/${script_name}/all_results.jld2 - сводные данные")
println(" • plots/${script_name}/ - все графики")
println(" • data/${script_name}/all_plots.jld2 - объекты графиков")
```

5.20 Базовые эксперименты

5.20.1 Базовая модель (model_type = base)

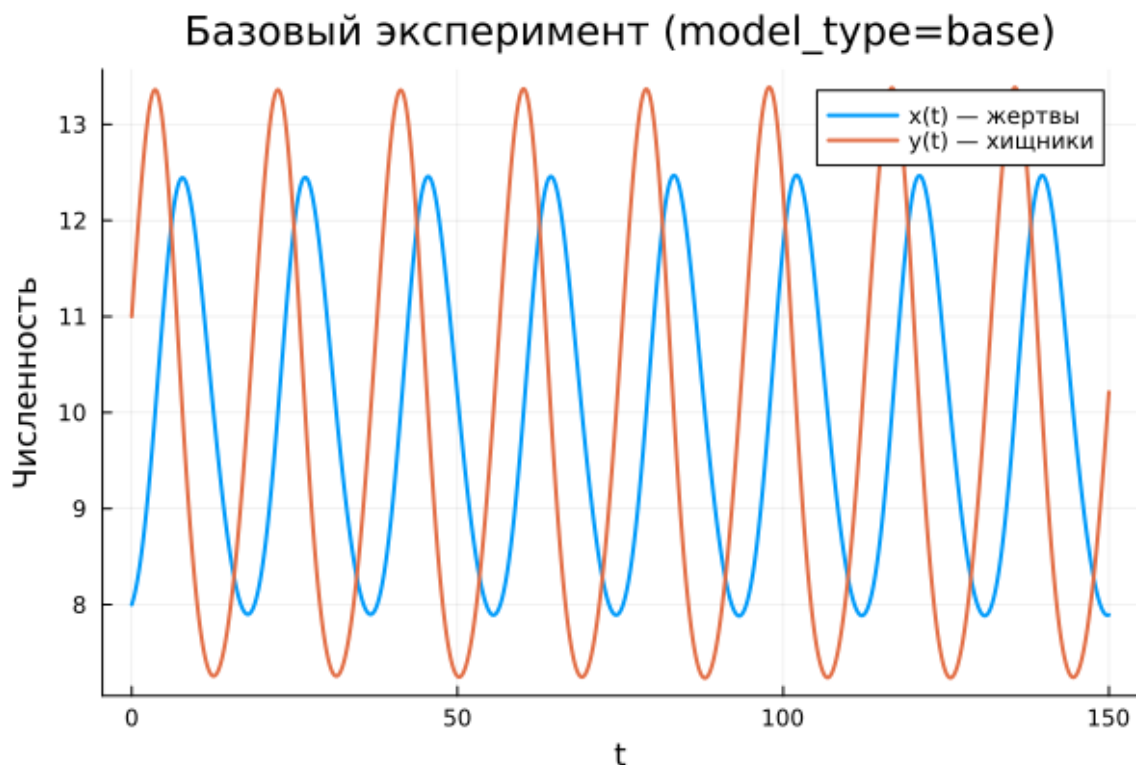


Рисунок 5.1: Базовая модель: зависимость $x(t)$, $y(t)$

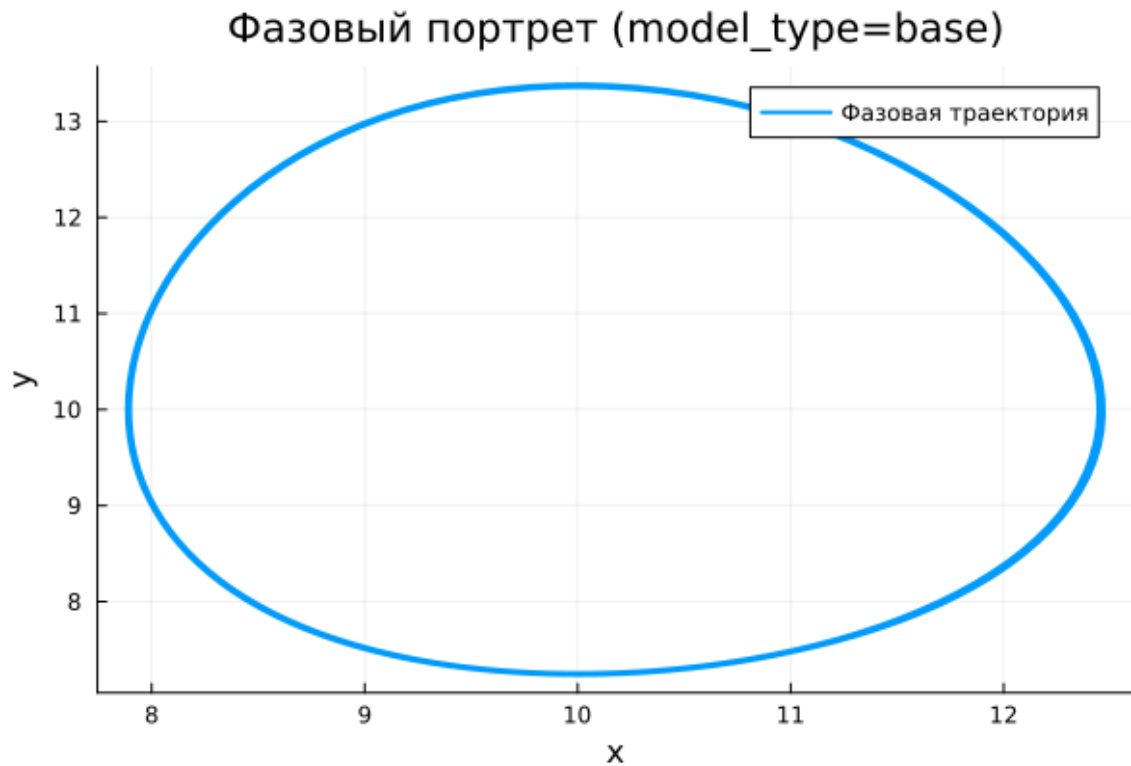


Рисунок 5.2: Базовая модель: фазовый портрет

В базовой постановке система демонстрирует устойчивые колебания без изменения амплитуды. Значения $x(t)$ и $y(t)$ изменяются циклически и повторяют поведение на протяжении всего временного интервала.

Такой режим соответствует идеализированной динамике без диссипации: энергия системы сохраняется, и переход к равновесию не происходит.

Фазовый портрет представляет собой замкнутую траекторию, что подтверждает периодичность процесса и отсутствие затухания.

5.20.2 Расширенная модель (model_type = extended)

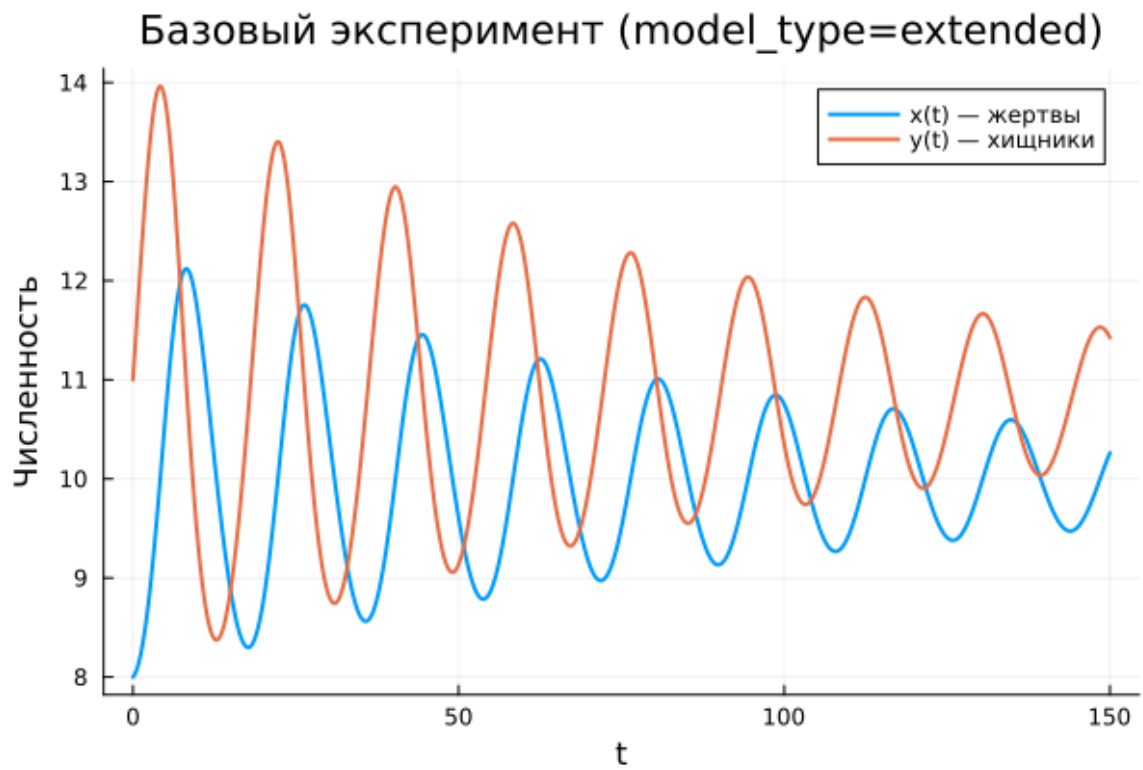


Рисунок 5.3: Расширенная модель: зависимость $x(t)$, $y(t)$

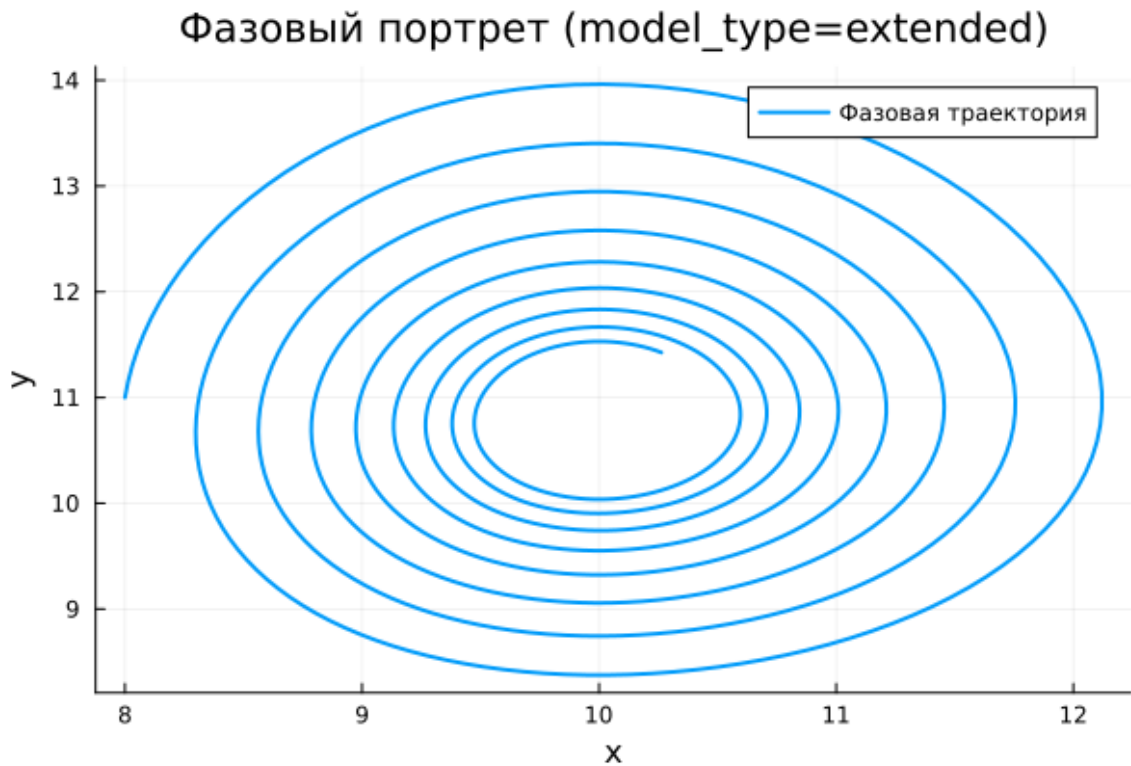


Рисунок 5.4: Расширенная модель: фазовый портрет

В расширенной модели динамика существенно меняется. На начальном этапе наблюдаются значительные колебания, однако со временем их амплитуда уменьшается.

Причиной является дополнительный член $-kx^2$, ограничивающий рост популяции жертв. В результате система утрачивает консервативные свойства и переходит к затухающим колебаниям.

Фазовая траектория принимает форму спирали, сходящейся к устойчивой точке. Это свидетельствует о наличии устойчивого равновесия.

Таким образом, расширенная модель приводит к стабилизации системы.

5.21 Параметрическое сканирование

5.21.1 Траектории $x(t)$ для различных параметров

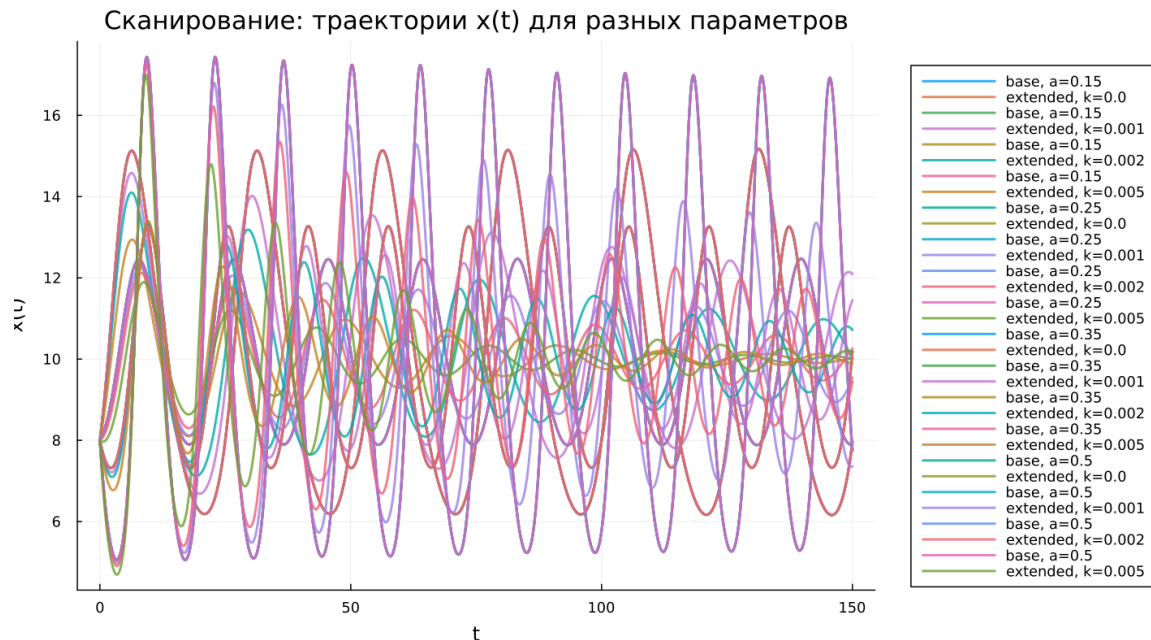


Рисунок 5.5: Сканирование траекторий $x(t)$

Проведено исследование влияния параметров на поведение системы.

В базовой модели варьирование параметра a изменяет характеристики колебаний (амплитуду и период), однако режим остаётся периодическим.

В расширенной модели изменение параметра k влияет на скорость затухания: при увеличении k система быстрее достигает равновесия.

Основные выводы: - базовая модель сохраняет колебательный режим

- расширенная модель приводит к затуханию

- параметры регулируют динамические характеристики

5.21.2 Траектории $y(t)$ для различных параметров

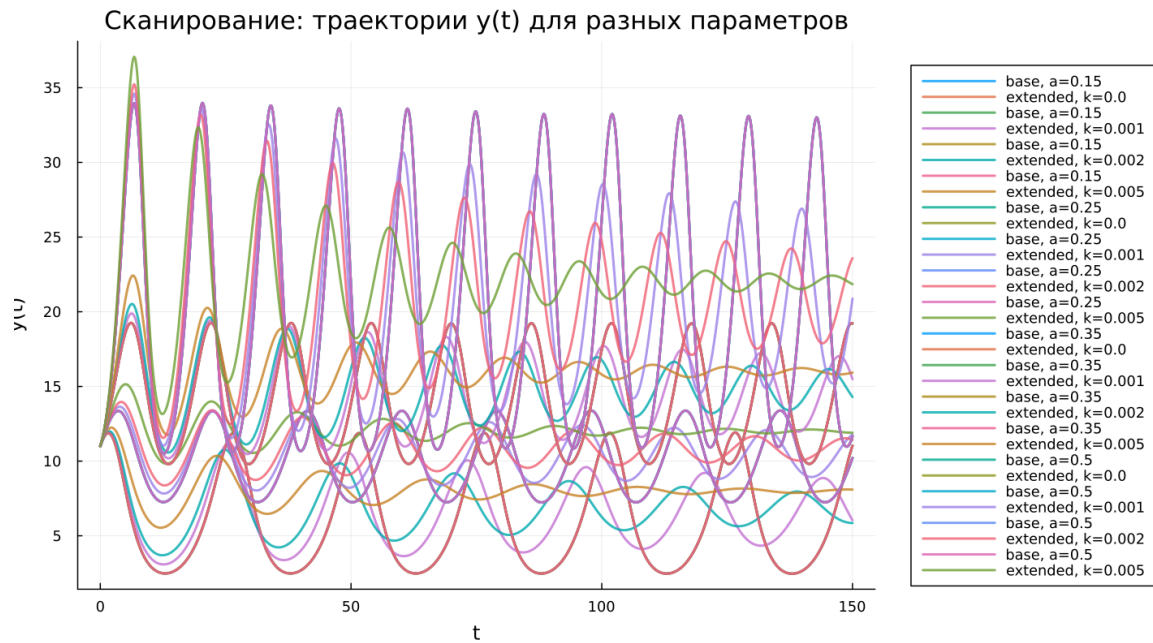


Рисунок 5.6: Сканирование траекторий $y(t)$

Анализ поведения $y(t)$ показывает аналогичную картину. В базовой модели сохраняются устойчивые колебания.

В расширенной модели наблюдается уменьшение амплитуды и выход на стационарное значение. Увеличение нелинейного параметра ускоряет этот процесс.

5.21.3 Фазовые траектории для различных параметров

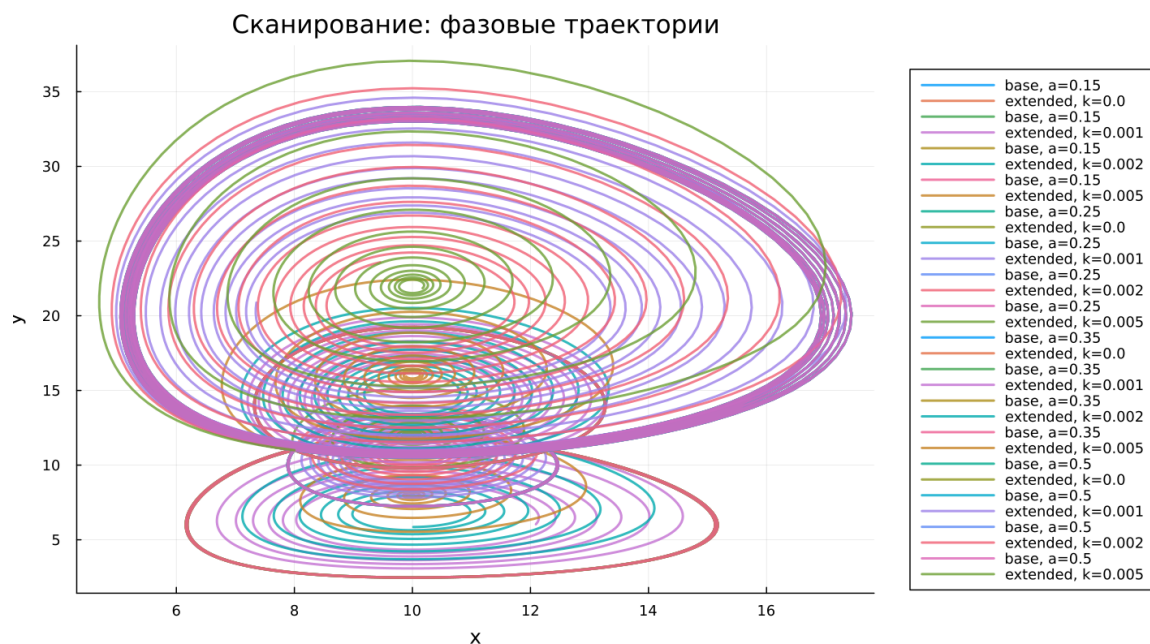


Рисунок 5.7: Сканирование фазовых траекторий

Фазовые диаграммы наглядно демонстрируют различия моделей.

Для базовой модели характерны замкнутые траектории, соответствующие устойчивым циклам.

Для расширенной модели траектории закручиваются к точке равновесия, что указывает на наличие устойчивого аттрактора.

Таким образом, изменение параметров не меняет природу моделей, а лишь влияет на количественные характеристики.

5.22 Анализ метрики norm_final

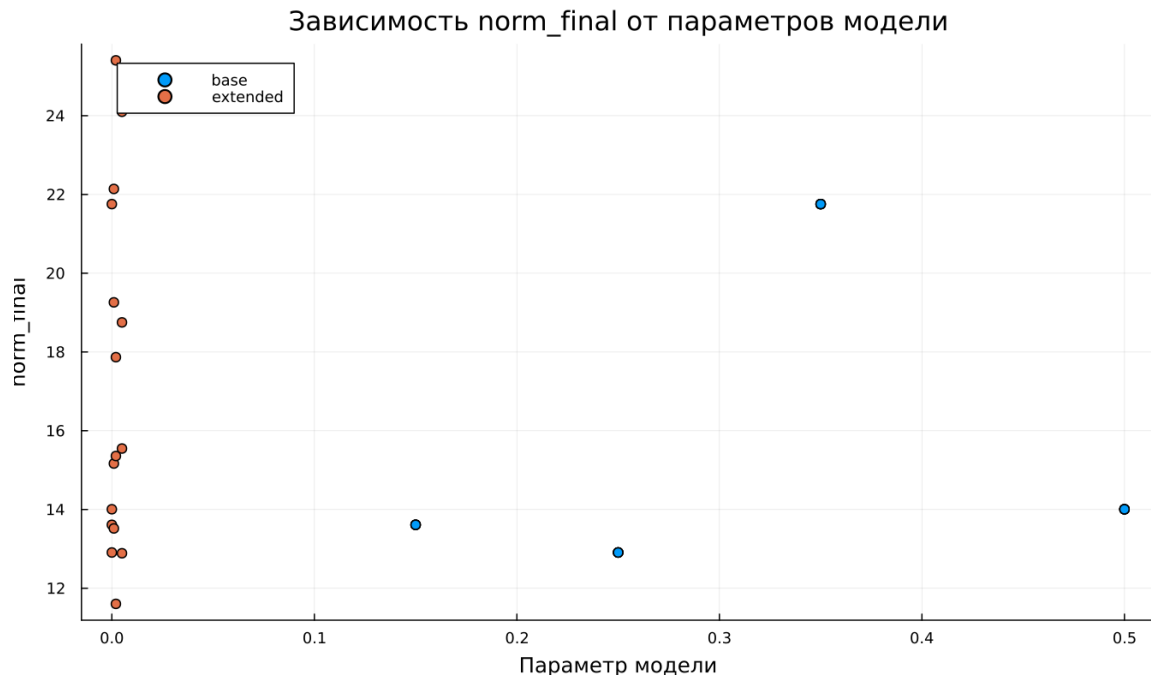


Рисунок 5.8: Зависимость norm_final

Рассматривалась величина:

$$\text{norm_final} = \sqrt{x(t_{\text{final}})^2 + y(t_{\text{final}})^2}$$

Для базовой модели значение метрики остаётся значительным, так как система продолжает колебаться и не достигает равновесия.

В расширенной модели метрика отражает положение стационарного состояния, поскольку колебания затухают.

5.23 Время вычислений

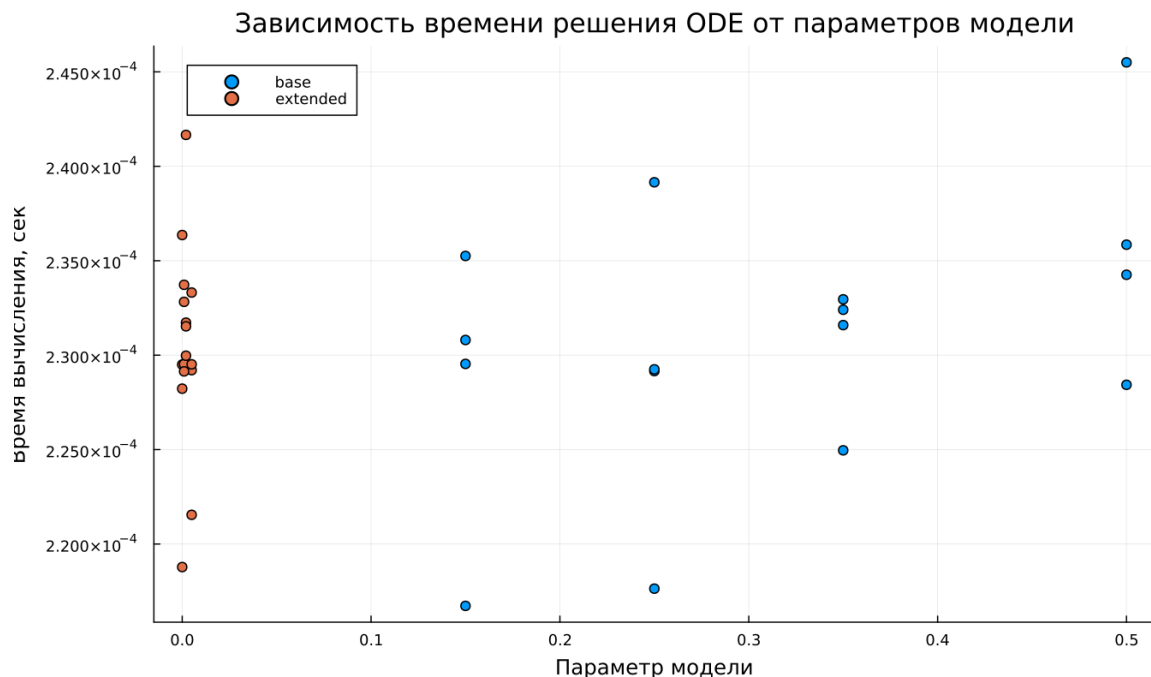


Рисунок 5.9: Зависимость времени вычисления

Анализ времени вычислений показывает, что численное решение выполняется быстро для обеих моделей.

Изменение параметров a и k практически не влияет на вычислительную сложность.

Добавление нелинейного члена не приводит к заметному увеличению затрат, что говорит о хорошей эффективности метода.

5.24 Выводы

1. Базовая модель описывает устойчивые периодические колебания без затухания

2. Расширенная модель приводит к установлению равновесия
3. Фазовые портреты подтверждают различие динамических режимов
4. Параметры a и k регулируют форму и скорость динамики
5. Метрика `norm_final` позволяет различить режимы поведения
6. Численные методы показывают высокую эффективность для обеих моделей

Список литературы

1. Модель Лотки-Вольтерры
2. Lotka-Volterra System